

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Drawing2D;
using System.Text;
namespace ImageOfStock
{
    public class CommonClass
    {
public static string GetGuid()
    {
        return System.Guid.NewGuid().ToString();
    }
public static double CalcuteSimpleMovingAvg(int r, int cycle, string field, string
target, double curValue, DataTable dataSource)
    {
        double sumValue = 0;
        if (r == cycle - 1)
        {
            for (int i = 0; i <= r - 1; i++)
            {
                sumValue += Convert.ToDouble(dataSource.Rows[i][target]);
            }
            sumValue += curValue;
            return sumValue / cycle;
        }
        else if (r > cycle - 1)
        {
            sumValue = Convert.ToDouble(dataSource.Rows[r - 1][field]) * cycle;
            sumValue -= Convert.ToDouble(dataSource.Rows[r - cycle][target]);
            sumValue += curValue;
            return sumValue / cycle;
        }
        else
        {
            return double.MinValue;
        }
    }
public static double CalculateExponentialMovingAvg(string field, string target,
int cycle, DataTable dataSource, int r)
    {
        DataRow dr = dataSource.Rows[r];
        double closeValue = Convert.ToDouble(dr[target]);
        double lastEMA = 0;
        double newEmaValue = 0;
        if (r > 0)
        {
            lastEMA = Convert.ToDouble(dataSource.Rows[r - 1][field]);
```

```
        newEmaValue = (closeValue * 2 + lastEMA * (cycle - 1)) / (cycle + 1);
    }
    return newEmaValue;
}

private static int panelID = 0;
public static int GetPanelID()
{
    return panelID++;
}

public static string GetTimeKey(DateTime dt)
{
    string month = dt.Month.ToString().Length == 1 ? "0" + dt.Month.ToString() :
dt.Month.ToString();
    string day = dt.Day.ToString().Length == 1 ? "0" + dt.Day.ToString() :
dt.Day.ToString();
    string hour = dt.Hour.ToString().Length == 1 ? "0" + dt.Hour.ToString() :
dt.Hour.ToString();
    string minute = dt.Minute.ToString().Length == 1 ? "0" + dt.Minute.ToString() :
dt.Minute.ToString();
    string second = dt.Second.ToString().Length == 1 ? "0" + dt.Second.ToString() :
dt.Second.ToString();
    return dt.Year + month + day + hour + minute + second;
}

public static DateTime GetDateTimeByTimeKey(string timeKey)
{
    int year = timeKey.Length >= 4 ? Convert.ToInt32(timeKey.Substring(0, 4)) :
1970;
    int month = timeKey.Length >= 6 ? Convert.ToInt32(timeKey.Substring(4, 2)) :
1;
    int day = timeKey.Length >= 8 ? Convert.ToInt32(timeKey.Substring(6, 2)) : 1;
    int hr = timeKey.Length >= 10 ? Convert.ToInt32(timeKey.Substring(8, 2)) : 0;
    int mn = timeKey.Length >= 12 ? Convert.ToInt32(timeKey.Substring(10, 2)) :
0;
    int sc = timeKey.Length >= 14 ? Convert.ToInt32(timeKey.Substring(12, 2)) :
0;
    DateTime dt = new DateTime(year, month, day, hr, mn, sc);
    return dt;
}

public static string GetCalenderFormatTimeKey(string value, IntervalType
interval)
{
    string timeKey = string.Empty;
    switch (interval)
    {
        case IntervalType.Year:
            timeKey = value.Substring(0, 4);
            break;
        case IntervalType.Month:
            timeKey = value.Substring(0, 4) + "/" + value.Substring(4, 2);
            break;
    }
}
```

```
        case IntervalType.Week:
        case IntervalType.Day:
            timeKey = value.Substring(0, 4) + "/" + value.Substring(4, 2) + "/" +
value.Substring(6, 2);
            break;
        case IntervalType.Minute:
            timeKey = value.Substring(8, 2) + ":" + value.Substring(10, 2);
            break;
        case IntervalType.Second:
            timeKey = value.Substring(8, 2) + ":" + value.Substring(10, 2) + ":" +
value.Substring(12, 2);
            break;
        }
        return timeKey;
    }
}

public static double GetLowValue(List<double> valueList)
{
    double low = 0;
    for (int i = 0; i < valueList.Count; i++)
    {
        if (i == 0)
        {
            low = valueList[i];
        }
        else
        {
            if (low > valueList[i])
            {
                low = valueList[i];
            }
        }
    }
    return low;
}

public static double GetHighValue(List<double> valueList)
{
    double high = 0;
    for (int i = 0; i < valueList.Count; i++)
    {
        if (i == 0)
        {
            high = valueList[i];
        }
        else
        {
            if (high < valueList[i])
            {
                high = valueList[i];
            }
        }
    }
}
```

```
    }
    return high;
}

public static int GetLowRecord(List<object[]> dicValues)
{
    double low = 0;
    int index = -1;
    for (int i = 0; i < dicValues.Count; i++)
    {
        int j = Convert.ToInt32(dicValues[i][0]);
        double value = Convert.ToDouble(dicValues[i][1]);
        if (i == 0)
        {
            index = j;
            low = value;
        }
        else
        {
            if (low > value)
            {
                index = j;
                low = value;
            }
        }
    }
    return index;
}

public static int GetHighRecord(List<object[]> dicValues)
{
    double high = 0;
    int index = -1;
    for (int i = 0; i < dicValues.Count; i++)
    {
        int j = Convert.ToInt32(dicValues[i][0]);
        double value = Convert.ToDouble(dicValues[i][1]);
        if (i == 0)
        {
            index = j;
            high = value;
        }
        else
        {
            if (high < value)
            {
                high = value;
                index = j;
            }
        }
    }
    return index;
}
```

```
}
public static string GetValueByDigit(double value, int digit)
{
    if (digit > 0)
    {
        StringBuilder sbFormat = new StringBuilder();
        string strValue = value.ToString();
        if (strValue.IndexOf(".") != -1)
        {
            sbFormat.Append(strValue.Substring(0, strValue.IndexOf(".") + 1));
            for (int i = 0; i < digit; i++)
            {
                int pos = strValue.IndexOf(".") + (i + 1);
                if (pos <= strValue.Length - 1)
                {
                    sbFormat.Append(strValue.Substring(pos, 1));
                }
                else
                {
                    sbFormat.Append("0");
                }
            }
        }
        else
        {
            sbFormat.Append(strValue + ".");
            for (int i = 0; i < digit; i++)
            {
                sbFormat.Append("0");
            }
        }
        return sbFormat.ToString();
    }
    return value.ToString();
}

public static GraphicsPath GetRoundRectangle(int cornerRadius, RectangleF rect)
{
    GraphicsPath roundedRect = new GraphicsPath();

    roundedRect.CloseFigure();
    return roundedRect;
}

public static int GridScale(double XMin, double XMax, int N, ref double SMin, ref
double Step, double m_fTick)
{
    int iNegScl;
    int iNml;
    double lfIniStep;
    double lfSclStep;
    double lfTmp;
```

```
double lfSMax;
int it;
int i;
int[] Steps = { 10, 12, 15, 16, 20, 25, 30, 40, 50, 60, 75, 80, 100, 120, 150 };
int iNS = Steps.Length;
if (XMin > XMax)
{
lfTmp = XMin;
XMin = XMax;
XMax = lfTmp;
}
if (XMin == XMax)
XMax = XMin == 0.0 ? 1.0 : XMin + Math.Abs(XMin) / 10.0;
if (XMax <= 0)
{
iNegSc1 = 1;
lfTmp = XMin;
XMin = -XMax;
XMax = -lfTmp;
}
else
iNegSc1 = 0;
if (N < 2)
N = 2;
iNm1 = N - 1;
for (it = 0; it < 3; it++)
{
lfIniStep = (XMax - XMin) / iNm1;
lfSc1Step = lfIniStep;
int pow10 = 0;
for (; lfSc1Step < 10.0; lfSc1Step *= 10.0) pow10--;
for (; lfSc1Step > 100.0; lfSc1Step /= 10.0) pow10++;
for (i = 0; i < iNS && lfSc1Step > Steps[i]; i++) ;
do
{
Step = Steps[i] * Math.Pow(10, (double)pow10);
if (m_fTick != 0.0)
{
Step = Math.Floor(Step / m_fTick) * m_fTick;
}
SMin = Math.Floor(XMin / Step) * Step;
lfSMax = SMin + iNm1 * Step;
if (XMax <= lfSMax)
{
if (iNegSc1 == 1)
SMin = -lfSMax;
Step *= iNm1 / (N - 1);
return 1;
}
i++;
}
```

```

    }
    while (i < iNS);
    iNm1 *= 2;
    }
    return 0;
}

}

public class ChartPanel
{
public ChartPanel()
{
    infoBombField = CommonClass.GetGuid();
    panelBorder_Pen.DashStyle = DashStyle.Dash;
    grid_Pen.DashStyle = DashStyle.Dash;
}
~ChartPanel()
{
    if (bgBrush != null) { bgBrush.Dispose(); }
    if (xtip_Brush != null) { xtip_Brush.Dispose(); }
    if (xTipFont_Brush != null) { xTipFont_Brush.Dispose(); }
    if (xTipFont_Pen != null) { xTipFont_Pen.Dispose(); }
    if (leftyTip_Brush != null) { leftyTip_Brush.Dispose(); }
    if (leftyTipFont_Brush != null) { leftyTipFont_Brush.Dispose(); }
    if (leftTipFont_Pen != null) { leftTipFont_Pen.Dispose(); }
    if (rightyTip_Brush != null) { rightyTip_Brush.Dispose(); }
    if (rightyTipFont_Brush != null) { rightyTipFont_Brush.Dispose(); }
    if (panelBorder_Pen != null) { panelBorder_Pen.Dispose(); }
    if (titleFont_Brush != null) { titleFont_Brush.Dispose(); }
    if (grid_Pen != null) { grid_Pen.Dispose(); }
    if (coordinateXFont_Brush != null) { coordinateXFont_Brush.Dispose(); }
    if (leftYFont_Brush != null) { leftYFont_Brush.Dispose(); }
    if (rightYFont_Brush != null) { rightYFont_Brush.Dispose(); }
    if (rightyTipFont_Pen != null) { rightyTipFont_Pen.Dispose(); }
    if (xScalePen != null) { xScalePen.Dispose(); }
    if (leftScalePen != null) { leftScalePen.Dispose(); }
    if (rightScalePen != null) { rightScalePen.Dispose(); }
    if (crossHair_Pen != null) { crossHair_Pen.Dispose(); }
}

    private Pen crossHair_Pen = new Pen(Color.White);
public Pen CrossHair_Pen
{
    get { return crossHair_Pen; }
    set { crossHair_Pen = value; }
}

private int gridInterval = 3;
public int GridInterval
{
    get { return gridInterval; }
    set { gridInterval = value; }
}
private string infoBombField = string.Empty;

```

```
public string InfoBombField
{
    get { return infoBombField; }
    set { infoBombField = value; }
}
private Color infoBombColor = Color.White;
public Color InfoBombColor
{
    get { return infoBombColor; }
    set { infoBombColor = value; }
}
private Color infoBombSelectedColor = Color.FromArgb(255, 255, 153);
public Color InfoBombSelectedColor
{
    get { return infoBombSelectedColor; }
    set { infoBombSelectedColor = value; }
}
private Color infoBombTipColor = Color.FromArgb(255, 255, 153);
public Color InfoBombTipColor
{
    get { return infoBombTipColor; }
    set { infoBombTipColor = value; }
}
private Color infoBombTipTextColor = Color.Black;
public Color InfoBombTipTextColor
{
    get { return infoBombTipTextColor; }
    set { infoBombTipTextColor = value; }
}
private Dictionary<string, List<SignalSeries>> signalSeriesDic = new
Dictionary<string, List<SignalSeries>>();
public Dictionary<string, List<SignalSeries>> SignalSeriesDic
{
    get { return signalSeriesDic; }
    set { signalSeriesDic = value; }
}
private List<TitleField> titleFieldList = new List<TitleField>();
public List<TitleField> TitleFieldList
{
    get { return titleFieldList; }
    set { titleFieldList = value; }
}
private bool userDefinedTitle = false;
public bool UserDefinedTitle
{
    get { return userDefinedTitle; }
    set { userDefinedTitle = value; }
}
private int digit = 2;
public int Digit
```



```
{
    get { return digit; }
    set { digit = value; }
}
private int panelID;
public int PanelID
{
    get { return panelID; }
    set { panelID = value; }
}
private int verticalPercent = 0;
public int VerticalPercent
{
    get { return verticalPercent; }
    set { verticalPercent = value; }
}
private double yScaleTick = 0.01;
public double YScaleTick
{
    get { return yScaleTick; }
    set { yScaleTick = value; }
}
private string sellSignalField = string.Empty;
[Browsable(false)]
public string SellSignalField
{
    get { return sellSignalField; }
    set { sellSignalField = value; }
}
private string panelTitle = string.Empty;
public string PanelTitle
{
    get { return panelTitle; }
    set { panelTitle = value; }
}
private bool showGrid = true;
public bool ShowGrid
{
    get { return showGrid; }
    set { showGrid = value; }
}
private double maxValue;
[Browsable(false)]
public double MaxValue
{
    get { return maxValue; }
    set { maxValue = value; }
}
private double minValue;
[Browsable(false)]
```

```
public double MinValue
{
    get { return minValue; }
    set { minValue = value; }
}

private IntervalType interval = IntervalType.Day;
[Browsable(true)]
public IntervalType Interval
{
    get { return interval; }
    set { interval = value; }
}

private Rectangle rectPanel = new Rectangle();
public Rectangle Bounds
{
    get { return rectPanel; }
    set { rectPanel = value; }
}

private List<CandleSeries> candleSeriesList = new List<CandleSeries>();
public List<CandleSeries> CandleSeriesList
{
    get { return candleSeriesList; }
    set { candleSeriesList = value; }
}

private List<HistogramSeries> historamSeriesList = new List<HistogramSeries>();
public List<HistogramSeries> HistoramSeriesList
{
    get { return historamSeriesList; }
    set { historamSeriesList = value; }
}

private List<TrendLineSeries> trendLineSeriesList = new List<TrendLineSeries>();
public List<TrendLineSeries> TrendLineSeriesList
{
    get { return trendLineSeriesList; }
    set { trendLineSeriesList = value; }
}

private List<string> yScaleField = new List<string>();
public List<string> YScaleField
{
    get { return yScaleField; }
    set { yScaleField = value; }
}

private Brush bgBrush = new SolidBrush(Color.Black);
public Brush BgBrush
{
    get { return bgBrush; }
    set { bgBrush = value; }
}

private Pen xScalePen = new Pen(Color.Red);
public Pen XScalePen
```

```
{
    get { return xScalePen; }
    set { xScalePen = value; }
}
private Pen leftScalePen = new Pen(Color.Red);
public Pen LeftScalePen
{
    get { return leftScalePen; }
    set { leftScalePen = value; }
}
private Pen rightScalePen = new Pen(Color.Red);
public Pen RightScalePen
{
    get { return rightScalePen; }
    set { rightScalePen = value; }
}
private Brush xtip_Brush = new SolidBrush(Color.FromArgb(100, Color.Red));
public Brush Xtip_Brush
{
    get { return xtip_Brush; }
    set { xtip_Brush = value; }
}
private Pen xTipFont_Pen = new Pen(Color.White);
public Pen XTipFont_Pen
{
    get { return xTipFont_Pen; }
    set { xTipFont_Pen = value; }
}
private Brush xTipFont_Brush = new SolidBrush(Color.White);
public Brush XTipFont_Brush
{
    get { return xTipFont_Brush; }
    set { xTipFont_Brush = value; }
}
private Font xTipFont = new Font("New Times Roman", 10, FontStyle.Bold);
public Font XTipFont
{
    get { return xTipFont; }
    set { xTipFont = value; }
}
private Brush leftyTip_Brush = new SolidBrush(Color.FromArgb(100, Color.Red));
public Brush LeftyTip_Brush
{
    get { return leftyTip_Brush; }
    set { leftyTip_Brush = value; }
}
private Pen leftTipFont_Pen = new Pen(Color.White);
public Pen LeftTipFont_Pen
{
    get { return leftTipFont_Pen; }
```

```
        set { leftTipFont_Pen = value; }
    }
private Brush leftyTipFont_Brush = new SolidBrush(Color.White);
public Brush LeftyTipFont_Brush
{
    get { return leftyTipFont_Brush; }
    set { leftyTipFont_Brush = value; }
}
private Font leftyTipFont = new Font("New Times Roman", 10, FontStyle.Bold);
public Font LeftyTipFont
{
    get { return leftyTipFont; }
    set { leftyTipFont = value; }
}
private Brush rightyTip_Brush = new SolidBrush(Color.FromArgb(100, Color.Red));
public Brush RightyTip_Brush
{
    get { return rightyTip_Brush; }
    set { rightyTip_Brush = value; }
}
private Brush rightyTipFont_Brush = new SolidBrush(Color.White);
public Brush RightyTipFont_Brush
{
    get { return leftyTipFont_Brush; }
    set { leftyTipFont_Brush = value; }
}
private Pen rightyTipFont_Pen = new Pen(Color.White);
public Pen RightyTipFont_Pen
{
    get { return rightyTipFont_Pen; }
    set { rightyTipFont_Pen = value; }
}
private Font rightyTipFont = new Font("New Times Roman", 10, FontStyle.Bold);
public Font RightyTipFont
{
    get { return leftyTipFont; }
    set { leftyTipFont = value; }
}
private Pen panelBorder_Pen = new Pen(Color.SkyBlue);
public Pen BorderPen
{
    get { return panelBorder_Pen; }
    set { panelBorder_Pen = value; }
}
private Brush titleFont_Brush = new SolidBrush(Color.White);
public Brush TitleFont_Brush
{
    get { return titleFont_Brush; }
    set { titleFont_Brush = value; }
}
```

```
private Font titleFont = new Font("New Times Roman", 9);
public Font TitleFont
{
    get { return titleFont; }
    set { titleFont = value; }
}
private Pen grid_Pen = new Pen(Color.FromArgb(100, Color.Red));
public Pen Grid_Pen
{
    get { return grid_Pen; }
    set { grid_Pen = value; }
}
private Brush coordinateXFont_Brush = new SolidBrush(Color.White);
public Brush CoordinateXFont_Brush
{
    get { return coordinateXFont_Brush; }
    set { coordinateXFont_Brush = value; }
}
private Font coordinateXFont = new Font("New Times Roman", 9);
public Font CoordinateXFont
{
    get { return coordinateXFont; }
    set { coordinateXFont = value; }
}
private Brush leftYFont_Brush = new SolidBrush(Color.White);
public Brush LeftYFont_Brush
{
    get { return leftYFont_Brush; }
    set { leftYFont_Brush = value; }
}
private Font leftYFont = new Font("New Times Roman", 9);
public Font LeftYFont
{
    get { return leftYFont; }
    set { leftYFont = value; }
}
private Brush rightYFont_Brush = new SolidBrush(Color.White);
public Brush RightYFont_Brush
{
    get { return rightYFont_Brush; }
    set { rightYFont_Brush = value; }
}
private Font rightYFont = new Font("New Times Roman", 9);
public Font RightYFont
{
    get { return rightYFont; }
    set { rightYFont = value; }
}
private float titleHeight = 30;
[Browsable(true)]
```

```

public float TitleHeight
{
    get { return titleHeight; }
    set { titleHeight = value; }
}
private int scaleX_Height = 25;
public int ScaleX_Height
{
    get { return scaleX_Height; }
    set { scaleX_Height = value; }
}
}
foreach (int panelID in infoBombDic.Keys)
{
    ChartPanel chartPanel = this.dicChartPanel[panelID];
    string bombInfoText = infoBombDic[panelID];
    Color bgColor = chartPanel.InfoBombTipColor;
    Color strColor = chartPanel.InfoBombTipTextColor;
    Pen tipBorderPen = new Pen(bgColor);
    tipBorderPen.DashStyle = DashStyle.Dot;
    Brush bgBrush = new SolidBrush(bgColor);
    Brush strBrush = new SolidBrush(strColor);
    SizeF sizeF = g.MeasureString(bombInfoText, new Font("", 10));
    g.DrawRectangle(tipBorderPen, this.leftPixSpace + 1,
(int)chartPanel.Bounds.Y + chartPanel.TitleHeight + 5, (int)sizeF.Width,
(int)sizeF.Height);
    g.FillRectangle(bgBrush, new Rectangle(this.leftPixSpace + 2,
(int)chartPanel.Bounds.Y + (int)chartPanel.TitleHeight + 6, (int)sizeF.Width -
1, (int)sizeF.Height - 1));
    g.DrawString(bombInfoText, new Font("", 10), strBrush, this.leftPixSpace +
2, chartPanel.Bounds.Y + chartPanel.TitleHeight + 7);
    tipBorderPen.Dispose();
    bgBrush.Dispose();
    strBrush.Dispose();
}
}
public void DrawValuePanel(Graphics g)
{
    if (selectedObject != null)
    {
        if (vp_index > 0 && vp_index <= LastVisibleRecord - 1)
        {
            Point mouseP = GetCrossHairPoint();
            int digit = 2;
            IntervalType curIntervalType = IntervalType.Day;
            foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
            {
                if (mouseP.Y >= chartPanel.Bounds.Y && mouseP.Y <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height)
                {

```

```

if (GetWorkSpaceY(chartPanel.PanelID) > 0)
{
    digit = chartPanel.Digit;
    curIntervalType = chartPanel.Interval;
}
}

DataRow dr = this.dtAllMsg.Rows[vp_index];
Point mp = new Point(GetCrossHairPoint().X + 10, GetCrossHairPoint().Y);
string timeKey = timekeyField + ":" +
CommonClass.GetCalenderFormatTimeKey(dr[timekeyField].ToString(),
curIntervalType);
Font tipFont = new Font("New Times Roman", 10, FontStyle.Bold);
SizeF timeKeySize = g.MeasureString(timeKey, tipFont);
double pWidth = 0;
double pHeight = 0;
List<double> wList = new List<double>();
StringBuilder sbValue = new StringBuilder();
sbValue.Append(timeKey + "\r\n");
Color pColor = Color.Turquoise;
if (selectedObject is CandleSeries)
{
    CandleSeries cs = selectedObject as CandleSeries;
    double open = 0;
    Double.TryParse(dr[cs.OpenField].ToString(), out open);
    double high = 0;
    Double.TryParse(dr[cs.HighField].ToString(), out high);
    double low = 0;
    Double.TryParse(dr[cs.LowField].ToString(), out low);
    double close = 0;
    Double.TryParse(dr[cs.CloseField].ToString(), out close);
    string strOpen = cs.OpenField + ":" + CommonClass.GetValueByDigit(open, digit);
    sbValue.Append(strOpen + "\r\n");
    SizeF openSize = g.MeasureString(strOpen, tipFont);
    string strHigh = cs.HighField + ":" + CommonClass.GetValueByDigit(high, digit);
    sbValue.Append(strHigh + "\r\n");
    SizeF highSize = g.MeasureString(strHigh, tipFont);
    string strLow = cs.LowField + ":" + CommonClass.GetValueByDigit(low, digit);
    sbValue.Append(strLow + "\r\n");
    SizeF lowSize = g.MeasureString(strLow, tipFont);
    string strClose = cs.CloseField + ":" + CommonClass.GetValueByDigit(close,
digit);
    sbValue.Append(strClose);
    SizeF closeSize = g.MeasureString(strClose, tipFont);
    wList.AddRange(new double[] { timeKeySize.Width, openSize.Width,
highSize.Width, lowSize.Width, closeSize.Width });
    pWidth = CommonClass.GetHighValue(wList);
    pHeight = timeKeySize.Height + openSize.Height + highSize.Height +
lowSize.Height + closeSize.Height;
}

```

```

        else if (selectedObject is HistogramSeries)
        {
            HistogramSeries hs = selectedObject as HistogramSeries;
            double volumn = 0;
            Double.TryParse(dr[hs.Field].ToString(), out volumn);
            string strVolume = hs.Field + ":" + CommonClass.GetValueByDigit(volumn, digit);
            sbValue.Append(strVolume);
            SizeF volumeSize = g.MeasureString(strVolume, tipFont);
            wList.AddRange(new double[] { timeKeySize.Width, volumeSize.Width });
            pWidth = CommonClass.GetHighValue(wList);
            pHeight = timeKeySize.Height + volumeSize.Height;
            pColor = Color.Yellow;
        }
        else if (selectedObject is TrendLineSeries)
        {
            TrendLineSeries tls = selectedObject as TrendLineSeries;
            double lineValue = 0;
            Double.TryParse(dr[tls.Field].ToString(), out lineValue);
            string strLine = tls.DisplayName != null ? tls.DisplayName + ":" +
            CommonClass.GetValueByDigit(lineValue, digit) : tls.Field + ":" +
            CommonClass.GetValueByDigit(lineValue, digit);
            sbValue.Append(strLine);
            SizeF lineSize = g.MeasureString(strLine, tipFont);
            wList.AddRange(new double[] { timeKeySize.Width, lineSize.Width });
            pWidth = CommonClass.GetHighValue(wList);
            pHeight = timeKeySize.Height + lineSize.Height;
            pColor = tls.UpLineColor;
        }
        pWidth += 4;
        pHeight += 1;
        Rectangle rectP = new Rectangle(GetCrossHairPoint().X + 10,
        GetCrossHairPoint().Y, (int)pWidth, (int)pHeight);
        Brush pbgBrush = new SolidBrush(Color.FromArgb(100, Color.Black));
        Pen pPen = new Pen(pColor);
        Brush pBrush = new SolidBrush(pColor);
        g.FillRectangle(pbgBrush, rectP);
        g.DrawRectangle(pPen, rectP);
        g.DrawString(sbValue.ToString(), tipFont, pBrush, new
        PointF(GetCrossHairPoint().X + 10, GetCrossHairPoint().Y + 2));
        pbgBrush.Dispose();
        pPen.Dispose();
        pBrush.Dispose();
    }
}

#endregion
#region

private void ChartGraph_MouseWheel(object sender, MouseEventArgs e)
{

```



```
        if (e.Delta > 0)
        {
            ZoomIn(2);
        }
        else
        {
            ZoomOut(2);
        }
        this.setVisibleExtremeValue();
        ResetCrossOverRecord();
        PaintGraph();
        this.Focus();
    }
    private void ChartGraph_MouseMove(object sender, MouseEventArgs e)
    {
        vp_index = -1;
        PaintGraph();
        if (showCrossLine)
        {
            selectedObject = null;
        }
        if (selectedObject != null)
        {
            if (lastMouseMoveTime.AddTicks(1000000) < DateTime.Now)
            {
                PaintGraph();
            }
            lastMouseMoveTime = DateTime.Now;
            drawValuePanelFlag = true;
        }
        if (this.userResizePanel == null)
        {
            if (!showCrossLine && !(e.Button == MouseButtons.Left))
            {
                int pIndex = 0;
                foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
                {
                    pIndex++;
                    if (pIndex == dicChartPanel.Count)
                    {
                        break;
                    }
                    Rectangle resizeRect = new Rectangle(0, chartPanel.Bounds.Bottom - 2,
                    chartPanel.Bounds.Width, 4);
                    if (resizeRect.Contains(e.Location))
                    {
                        this.Cursor = Cursors.SizeNS;
                        goto OutLoop;
                    }
                }
            }
        }
    }
}
```

```
        if (this.Cursor == Cursors.SizeNS)
        {
            this.Cursor = Cursors.Default;
        }
    OutLoop: ;
}

}

private void ChartGraph_MouseUp(object sender, MouseEventArgs e)
{
    this.Cursor = Cursors.Default;
    Point mp = GetCrossHairPoint();
    if (canDragSeries)
    {
        if (!showCrossLine && selectedObject != null)
        {
            foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
            {
                if (mp.Y >= chartPanel.Bounds.Y && mp.Y <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height)
                {
                    if (selectedObject is TrendLineSeries)
                    {
                        TrendLineSeries tls = selectedObject as TrendLineSeries;
                        if (!chartPanel.TrendLineSeriesList.Contains(tls))
                        {
                            TitleField tfWaitToDrag = null;
                            foreach (ChartPanel cp in this.dicChartPanel.Values)
                            {
                                if (cp.TrendLineSeriesList.Contains(tls))
                                {
                                    cp.TrendLineSeriesList.Remove(tls);
                                    if (cp.YScaleField.Contains(tls.Field))
                                    {
                                        cp.YScaleField.Remove(tls.Field);
                                    }
                                }
                                foreach (TitleField tf in cp.TitleFieldList)
                                {
                                    if (!tf.MainFlag && tf.RelateSeriesField == tls.Field)
                                    {
                                        tfWaitToDrag = tf;
                                    }
                                }
                            }
                            if (tfWaitToDrag != null)
                            {
                                cp.TitleFieldList.Remove(tfWaitToDrag);
                            }
                            break;
                        }
                    }
                }
            }
        }
    }
}
```

```
        chartPanel.TrendLineSeriesList.Add(tls);
chartPanel.YScaleField.Add(tls.Field);
if (tfWaitToDrag != null)
{
    chartPanel.TitleFieldList.Add(tfWaitToDrag);
}
selectedObject = null;
RefreshGraph();
}
}
else if (selectedObject is HistogramSeries)
{
    HistogramSeries hs = selectedObject as HistogramSeries;
    if (!chartPanel.HistoramSeriesList.Contains(hs))
    {
        TitleField tfWaitToDrag = null;
        foreach (ChartPanel cp in this.dicChartPanel.Values)
        {
            if (cp.HistoramSeriesList.Contains(hs))
            {
                if (cp.YScaleField.Contains(hs.Field))
                {
                    cp.YScaleField.Remove(hs.Field);
                }
                cp.HistoramSeriesList.Remove(hs);
                foreach (TitleField tf in cp.TitleFieldList)
                {
                    if (!tf.MainFlag && tf.RelateSeriesField == hs.Field)
                    {
                        tfWaitToDrag = tf;
                    }
                }
            }
            if (tfWaitToDrag != null)
            {
                cp.TitleFieldList.Remove(tfWaitToDrag);
            }
            break;
        }
    }
    if (tfWaitToDrag != null)
    {
        chartPanel.TitleFieldList.Add(tfWaitToDrag);
    }
    chartPanel.HistoramSeriesList.Add(hs);
    chartPanel.YScaleField.Add(hs.Field);
    selectedObject = null;
    RefreshGraph();
}
}
else if (selectedObject is CandleSeries)
```

```
{
    CandleSeries cs = selectedObject as CandleSeries;
    if (!chartPanel.CandleSeriesList.Contains(cs))
    {
List<TitleField> waitToRemoveTfList = new List<TitleField>();
foreach (ChartPanel cp in this.dicChartPanel.Values)
{
    if (cp.CandleSeriesList.Contains(cs))
    {
        if (cp.YScaleField.Contains(cs.CloseField))
        {
            cp.YScaleField.Remove(cs.CloseField);
        }
        if (cp.YScaleField.Contains(cs.HighField))
        {
            cp.YScaleField.Remove(cs.HighField);
        }
        if (cp.YScaleField.Contains(cs.LowField))
        {
            cp.YScaleField.Remove(cs.LowField);
        }
        if (cp.YScaleField.Contains(cs.OpenField))
        {
            cp.YScaleField.Remove(cs.OpenField);
        }
        cp.CandleSeriesList.Remove(cs);
        foreach (TitleField tf in cp.TitleFieldList)
        {
            if (!tf.MainFlag)
            {
                if (tf.RelateSeriesField == cs.OpenField || tf.RelateSeriesField ==
cs.HighField
|| tf.RelateSeriesField == cs.LowField || tf.RelateSeriesField == cs.CloseField)
                {
waitToRemoveTfList.Add(tf);
                }
            }
        }
        foreach (TitleField tf in waitToRemoveTfList)
        {
            cp.TitleFieldList.Remove(tf);
        }
    }
}

chartPanel.CandleSeriesList.Add(cs);
chartPanel.YScaleField.Add(cs.OpenField);
chartPanel.YScaleField.Add(cs.HighField);
chartPanel.YScaleField.Add(cs.CloseField);
chartPanel.YScaleField.Add(cs.LowField);
foreach (TitleField tf in waitToRemoveTfList)
```

```
{
    chartPanel.TitleFieldList.Add(tf);
}
selectedObject = null;
RefreshGraph();
}
}
}
}
}
DragChartPanel();
}
private void ChartGraph_MouseDown(object sender, MouseEventArgs e)
{
    Point mp = GetCrossHairPoint();
    mouse_x = mp.X;
    mouse_y = mp.Y;
    if (e.Button == MouseButtons.Left)
    {
        crossHair_y = mp.Y;
        if (e.Clicks == 1)
        {
            this.crossOverIndex = this.GetCrossOverIndex();
            object obj = JudgeSelectedSeries(crossOverIndex, crossHair_y, true);
            if (obj != null && !showCrossLine && canDragSeries)
            {
                this.Cursor = Cursors.Cross;
            }
            PaintGraph();
        }
        else if (e.Clicks == 2)
        {
            this.ShowCrossHair = !this.ShowCrossHair;
            this.crossOverIndex = this.GetCrossOverIndex();
            selectedObject = null;
            PaintGraph();
        }
    }
    if (!showCrossLine)
    {
        int pIndex = 0;
        foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
        {
            pIndex++;
            if (pIndex == dicChartPanel.Count)
            {
                break;
            }
            Rectangle resizeRect = new Rectangle(0, chartPanel.Bounds.Bottom - 2,
```

```

chartPanel.Bounds.Width, 4);
    if (resizeRect.Contains(mp))
    {
        this.Cursor = Cursors.SizeNS;
        userResizePanel = chartPanel;
        PaintGraph();
        goto OutLoop;
    }
}

OutLoop: ;
}
else
{
    userResizePanel = null;
}
this.Focus();
}

protected override void OnPaint(PaintEventArgs e)
{
    base.OnPaint(e);
    PaintGraph(e);
    public void PaintGraph(PaintEventArgs e = null)
    {
        lock (refresh_lock)
        {
            BufferedGraphics myBuffer =
            BufferedGraphicsManager.Current.Allocate(this.CreateGraphics(),
            DisplayRectangle);
            Graphics g = myBuffer.Graphics;
            DrawBackGround(g);
            DrawTitle(g);
            DrawScale(g);
            DrawSeries(g);
            if (showCrossLine)
                DrawCrossLine(g);
            myBuffer.Render();
            myBuffer.Dispose();
        }
    }
}

protected override void WndProc(ref Message m)
{
    if (m.Msg == 0x0014)
        return;
    base.WndProc(ref m);
}

private void ChartGraph_SizeChanged(object sender, EventArgs e)
{
    if (this.Size.Width != 0 && this.Size.Height != 0)
    {

```

```
        ResizeGraph();
        RefreshGraph();
    }
}

private void ChartGraph_KeyUp(object sender, KeyEventArgs e)
{
    currentScrollStep = 1;
}

private void PicGraph_PreviewKeyDown(object sender, PreviewKeyDownEventArgs e)
{
    vp_index = -1;
    //if (processBarValue == 0)
    {
        bool flag = false;
        bool locateCrossHairFlag = false;
        switch (e.KeyData)
        {
            case Keys.Left:
                flag = true;
                if (showCrossLine)
                {
                    CrossHairScrollLeft();
                    locateCrossHairFlag = true;
                }
                else
                {
                    this.ScrollLeft(currentScrollStep);
                }
                break;
            case Keys.Right:
                flag = true;
                if (showCrossLine)
                {
                    CrossHairScrollRight();
                    locateCrossHairFlag = true;
                }
                else
                {
                    this.ScrollRight(currentScrollStep);
                }
                break;
            case Keys.Up:
                flag = true;
                this.ZoomIn(currentScrollStep);
                break;
            case Keys.Down:
                flag = true;
                this.ZoomOut(currentScrollStep);
                break;
```

```
}
if (flag)
{
    this.setVisibleExtremeValue();
    ResetCrossOverRecord();
    if (locateCrossHairFlag)
    {
        LocateCrossHair();
    }
    PaintGraph();
}
}
if (useScrollAddSpeed)
{
if (currentScrollStep < 40)
{
    currentScrollStep += 5;
}
}
else
{
currentScrollStep = 1;
}
this.Focus();
}

private void ChartGraph_HandleCreated(object sender, EventArgs e)
{
    Thread checkMouseMoveThread = new Thread(new
ThreadStart(checkMouseMoveLoop));
    checkMouseMoveThread.IsBackground = true;
    checkMouseMoveThread.Start();
}

public void ClearSelectObj()
{
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        foreach (CandleSeries cs in chartPanel.CandleSeriesList)
        {
            cs.HasSelect = false;
        }
        foreach (HistogramSeries hs in chartPanel.HistoramSeriesList)
        {
            hs.HasSelect = false;
        }
        foreach (TrendLineSeries tls in chartPanel.TrendLineSeriesList)
        {
            tls.HasSelect = false;
        }
    }
}
```



```
    }
    selectedObject = null;
}

public SignalSeries AddSignal(int panelID, string timeKey, SignalType st, Color
stColor, double value, bool canDrag)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        SignalSeries ss = new SignalSeries(value, st, stColor, canDrag);
        if (!chartPanel.SignalSeriesDic.ContainsKey(timeKey))
        {
            chartPanel.SignalSeriesDic[timeKey] = new List<SignalSeries>();
        }
        chartPanel.SignalSeriesDic[timeKey].Add(ss);
        return ss;
    }
    return null;
}

public void RemoveSignal(int panelID, string timeKey)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        if (chartPanel.SignalSeriesDic.ContainsKey(timeKey))
        {
            chartPanel.SignalSeriesDic.Remove(timeKey);
        }
    }
}

public object JudgeSelectedSeries(int curIndex, int mpY, bool setSelect)
{
    bool hasSelect = false;
    object obj = null;
    if (setSelect)
    {
        Point mp = GetCrossHairPoint();
        foreach (RectangleF titleRect in this.objectRectDic.Keys)
        {
            if (titleRect.Contains(mp))
            {
                hasSelect = true;
                obj = objectRectDic[titleRect];
                this.selectedObject = obj;
                break;
            }
        }
    }
}
```

```

    }
    if (firstVisibleRecord != 0 && LastVisibleRecord != 0)
    {
        foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
        {
            if (chartPanel.TrendLineSeriesList.Count > 0)
            {
                foreach (TrendLineSeries tls in chartPanel.TrendLineSeriesList)
                {
                    if (hasSelect)
                    {
                        if (setSelect)
                        {
                            if (obj != null && obj == tls)
                            {
                                tls.HasSelect = true;
                            }
                            else
                            {
                                tls.HasSelect = false;
                            }
                        }
                    }
                    else
                    {
                        if (curIndex > LastVisibleRecord - 1 ||
                            this.dtAllMsg.Rows[curIndex][tls.Field].ToString() == "")
                        {
                            if (setSelect)
                            {
                                tls.HasSelect = false;
                            }
                            continue;
                        }
                        double lineValue =
                            Convert.ToDouble(this.dtAllMsg.Rows[curIndex][tls.Field].ToString());
                        float scaleX = this.leftPixSpace + (curIndex + 2 - firstVisibleRecord) *
                            axisSpace - axisSpace / 2;
                        int topY = Convert.ToInt32(GetValueYPixel(chartPanel, lineValue));
                        Point crossHairP = GetCrossHairPoint();
                        int judgeTop = 0;
                        float judgeScaleX = scaleX;
                        if (crossHairP.X >= scaleX)
                        {
                            if (curIndex < this.LastVisibleRecord - 1 && this.dtAllMsg.Rows[curIndex +
                                1][tls.Field].ToString() != "")
                            {
                                double rightValue = Convert.ToDouble(this.dtAllMsg.Rows[curIndex +
                                    1][tls.Field].ToString());
                                judgeTop = Convert.ToInt32(GetValueYPixel(chartPanel, rightValue));

```

```

        if (judgeTop > chartPanel.Bounds.Y + chartPanel.Bounds.Height -
chartPanel.ScaleX_Height || judgeTop < chartPanel.Bounds.Y +
chartPanel.TitleHeight)
        {
            if (setSelect)
            {
                tls.HasSelect = false;
            }
            continue;
        }
    }
    else
    {
        judgeTop = topY;
    }
    else
    {
        judgeScaleX = scaleX - axisSpace;
        if (curIndex > 0 && this.dtAllMsg.Rows[curIndex - 1][tls.Field].ToString() != "")
        {
            double leftValue = Convert.ToDouble(this.dtAllMsg.Rows[curIndex -
1][tls.Field].ToString());
            judgeTop = Convert.ToInt32(GetValueYPixel(chartPanel, leftValue));
            if (judgeTop > chartPanel.Bounds.Y + chartPanel.Bounds.Height -
chartPanel.ScaleX_Height || judgeTop < chartPanel.Bounds.Y +
chartPanel.TitleHeight)
            {
                if (setSelect)
                {
                    tls.HasSelect = false;
                }
                continue;
            }
        }
    }
    else
    {
        judgeTop = topY;
    }
    Rectangle judgeRect = new Rectangle();
    if (judgeTop >= topY)
    {
        judgeRect = new Rectangle((int)judgeScaleX, topY, axisSpace, judgeTop - topY < 1 ?
1 : judgeTop - topY);
    }
    else
    {
        judgeRect = new Rectangle((int)judgeScaleX, judgeTop, axisSpace, topY - judgeTop
< 1 ? 1 : topY - judgeTop);
    }
}

```

```

    }
    if (judgeRect.Contains(crossHairP))
    {
    if (setSelect)
    {
        selectedObject = tls;
        tls.HasSelect = true;
    }
    obj = tls;
    hasSelect = true;
    }
    else
    {
    if (setSelect)
    {
        tls.HasSelect = false;
    }
    }
    }
    }
    if (chartPanel.HistoramSeriesList.Count > 0)
    {
    foreach (HistogramSeries hs in chartPanel.HistoramSeriesList)
    {
    if (hasSelect)
    {
        if (setSelect)
        {
        if (obj != null && obj == hs)
        {
            hs.HasSelect = true;
        }
        else
        {
            hs.HasSelect = false;
        }
        }
    }
    else
    {
        if (curIndex > LastVisibleRecord - 1 ||
this.dtAllMsg.Rows[curIndex][hs.Field].ToString() == "")
        {
            hs.HasSelect = false;
            continue;
        }
        double volumn =
Convert.ToDouble(this.dtAllMsg.Rows[curIndex][hs.Field].ToString());
        int topY = Convert.ToInt32(GetValueYPixel(chartPanel, volumn));

```

```
        int bottomY = Convert.ToInt32(GetValueYPixel(chartPanel, 0));
        if (volumn < 0)
        {
            topY = Convert.ToInt32(GetValueYPixel(chartPanel, 0));
            bottomY = Convert.ToInt32(GetValueYPixel(chartPanel, volumn));
        }
        if (topY >= chartPanel.Bounds.Y && bottomY <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height
&& mpY >= topY && mpY <= bottomY)
        {
            if (setSelect)
            {
                selectedObject = hs;
                hs.HasSelect = true;
            }
            obj = hs;
            hasSelect = true;
        }
        else
        {
            if (setSelect)
            {
                hs.HasSelect = false;
            }
        }
    }
}

    if (chartPanel.CandleSeriesList.Count > 0)
    {
        foreach (CandleSeries cs in chartPanel.CandleSeriesList)
        {
            if (hasSelect)
            {
                if (setSelect)
                {
                    if (obj != null && obj == cs)
                    {
                        cs.HasSelect = true;
                    }
                }
                else
                {
                    cs.HasSelect = false;
                }
            }
        }
    }
    else
    {
        if (curIndex > LastVisibleRecord - 1
|| this.dtAllMsg.Rows[curIndex][cs.HighField].ToString() == "")
```

```

|| this.dtAllMsg.Rows[curIndex][cs.LowField].ToString() == "")
    {
if (setSelect)
{
    cs.HasSelect = false;
}
continue;
    }
    double highValue =
Convert.ToDouble(this.dtAllMsg.Rows[curIndex][cs.HighField].ToString());
    double lowValue =
Convert.ToDouble(this.dtAllMsg.Rows[curIndex][cs.LowField].ToString());
    int topY = Convert.ToInt32(GetValueYPixel(chartPanel, highValue));
    int bottomY = Convert.ToInt32(GetValueYPixel(chartPanel, lowValue));
    if (topY >= chartPanel.Bounds.Y && bottomY <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height
&& mpY >= topY && mpY <= bottomY)
    {
if (setSelect)
{
    cs.HasSelect = true;
    selectedObject = cs;
}
obj = cs;
hasSelect = true;
    }
    else
    {
if (setSelect)
{
    cs.HasSelect = false;
}
    }
}
    }
    }
    }
    if (obj == null)
    {
selectedObject = null;
    }
    return obj;
}
public void ResizeGraph()
{
    int locationY = 0;
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
chartPanel.Bounds = new Rectangle(0, locationY, this.Width,

```

```
Convert.ToInt32((double)chartPanel.VerticalPercent / 100 * this.Height));
    locationY += Convert.ToInt32((double)chartPanel.VerticalPercent / 100 *
this.Height);
}
}
public void DragChartPanel()
{
    if (userResizePanel != null)
    {
        Point mp = GetCrossHairPoint();
        ChartPanel nextCP = null;
        bool rightP = false;
        foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
        {
            if (rightP)
            {
                nextCP = chartPanel;
                break;
            }
            if (chartPanel == userResizePanel)
            {
                rightP = true;
            }
        }
        int originalVP = userResizePanel.VerticalPercent;
        if (userResizePanel.Bounds.Contains(mp))
        {
            userResizePanel.VerticalPercent = Convert.ToInt32(((double)mp.Y -
(double)userResizePanel.Bounds.Top) / (double)this.Height * 100);
            if (userResizePanel.VerticalPercent < 1)
            {
                userResizePanel.VerticalPercent = 1;
            }
            if (nextCP != null)
            {
                nextCP.VerticalPercent += originalVP - userResizePanel.VerticalPercent;
            }
        }
        else
        {
            if (nextCP != null && nextCP.Bounds.Contains(mp))
            {
                userResizePanel.VerticalPercent = Convert.ToInt32(((double)mp.Y -
(double)userResizePanel.Bounds.Top) / (double)this.Height * 100);
                if (userResizePanel.VerticalPercent >= originalVP + nextCP.VerticalPercent)
                {
                    userResizePanel.VerticalPercent -= 1;
                }
                nextCP.VerticalPercent = originalVP + nextCP.VerticalPercent -
userResizePanel.VerticalPercent;
            }
        }
    }
}
```

```

    }
}
userResizePanel = null;
ResizeGraph();
PaintGraph();
}
}

public void LocateCrossHair()
{
    if (this.dtAllMsg.Rows.Count > 0)
    {
        foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
        {
            if (crossHair_y >= chartPanel.Bounds.Y && crossHair_y <= chartPanel.Bounds.Y
+ chartPanel.Bounds.Height)
            {
                if (GetWorkSpaceY(chartPanel.PanelID) > 0)
                {
                    if (this.crossOverIndex >= 0 && this.crossOverIndex < this.dtAllMsg.Rows.Count)
                    {
                        if (chartPanel.CandleSeriesList.Count > 0)
                        {
                            double closeValue =
Convert.ToDouble(this.dtAllMsg.Rows[this.crossOverIndex][chartPanel.CandleSerie
sList[0].CloseField]);
                            crossHair_y = Convert.ToInt32(GetValueYPixel(chartPanel, closeValue));
                            return;
                        }
                        if (chartPanel.HistoramSeriesList.Count > 0)
                        {
                            double volumn =
Convert.ToDouble(this.dtAllMsg.Rows[this.crossOverIndex][chartPanel.HistoramSer
iesList[0].Field]);
                            crossHair_y = Convert.ToInt32(GetValueYPixel(chartPanel, volumn));
                            return;
                        }
                        if (chartPanel.TrendLineSeriesList.Count > 0)
                        {
                            double lineValue =
Convert.ToDouble(this.dtAllMsg.Rows[this.crossOverIndex][chartPanel.TrendLineSe
riesList[0].Field]);
                            crossHair_y = Convert.ToInt32(GetValueYPixel(chartPanel, lineValue));
                            return;
                        }
                    }
                }
            }
        }
    }
}

```



```
}  
public void CrossHairScrollLeft()  
{  
    int x = 1;  
    int currentRecord = this.crossOverIndex;  
    this.crossOverIndex = currentRecord - x;  
    if (crossOverIndex < 0)  
    {  
        crossOverIndex = 0;  
    }  
    if (currentRecord < this.firstVisibleRecord)  
    {  
        ScrollLeft(x);  
    }  
}  
  
public void CrossHairScrollRight()  
{  
    int currentRecord = this.crossOverIndex;  
    this.crossOverIndex = currentRecord + 1;  
    int maxRecord = GetMaxVisibleRecord();  
    if (this.dtAllMsg.Rows.Count < maxRecord)  
    {  
        if (crossOverIndex >= maxRecord-1)  
        {  
            crossOverIndex = maxRecord - 1;  
        }  
        if (currentRecord >= this.LastVisibleRecord - 1)  
        {  
            ScrollRight(1);  
        }  
    }  
}  
public void SetCandleBuySellField(string candleName, string buyField, string  
sellField)  
{  
    foreach (ChartPanel cp in this.dicChartPanel.Values)  
    {  
        foreach (CandleSeries cs in cp.CandleSeriesList)  
        {  
            if (cs.CandleName == candleName)  
            {  
                cs.IndBuySellField[0] = buyField;  
                cs.IndBuySellField[1] = sellField;  
                break;  
            }  
        }  
    }  
}
```

```
public void SetInfoBombText(int panelID, string info, DateTime dt)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        string field = chartPanel.InfoBombField;
        if (!this.dtAllMsg.Columns.Contains(field))
        {
            this.dtAllMsg.Columns.Add(field);
        }
        SetValue(field, info, dt);
    }
}

public void SetIntervalType(int panelID, IntervalType intervalType)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].Interval = intervalType;
    }
}

public void SetInfoBombStyle(int panelID, Color bgColor, Color
selectedColor, Color tipBgColor, Color tipTextColor)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        chartPanel.InfoBombColor = bgColor;
        chartPanel.InfoBombSelectedColor = selectedColor;
        chartPanel.InfoBombTipColor = tipBgColor;
        chartPanel.InfoBombTipTextColor = tipTextColor;
    }
}

public void SetCandleBuySellStyle(string candleName, string buyText, string
sellText, Color buyColor, Color sellColor, Font bsFont)
{
    foreach (ChartPanel cp in this.dicChartPanel.Values)
    {
        foreach (CandleSeries cs in cp.CandleSeriesList)
        {
            if (cs.CandleName == candleName)
            {
                cs.BuyText = buyText;
                cs.SellText = sellText;
                cs.BuyColor = buyColor;
                cs.SellColor = sellColor;
                cs.BsFont = bsFont;
                break;
            }
        }
    }
}
```

```
    }
}

}

public void SetTick(int panelID, double tick)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].YScaleTick = tick;
    }
}

public void SetDigit(int panelID, int digit)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].Digit = digit;
    }
}

public void SetScrollStep(int leftStep, int rightStep)
{
    this.scrollLeftStep = leftStep;
    this.scrollRightStep = rightStep;
}

public void SetTitle(int panelID, string title)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].PanelTitle = title;
    }
}

public void SetGridInterval(int panelID, int gridInterval)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].GridInterval = gridInterval;
    }
}

public void SetCrossHairStyle(int panelID, Color crossHairColor, int weight)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        chartPanel.CrossHair_Pen.Dispose();
        chartPanel.CrossHair_Pen = new Pen(crossHairColor, weight);
    }
}
```

```
    }  
}
```

```
public void SetXTipStyle(int panelID, Color backColor, Color fontColor, Font  
xTipFont)  
{  
    if (this.dicChartPanel.ContainsKey(panelID))  
    {  
        this.dicChartPanel[panelID].Xtip_Brush.Dispose();  
        this.dicChartPanel[panelID].Xtip_Brush = new SolidBrush(backColor);  
        this.dicChartPanel[panelID].XTipFont_Brush.Dispose();  
        this.dicChartPanel[panelID].XTipFont_Brush = new SolidBrush(fontColor);  
        this.dicChartPanel[panelID].XTipFont = xTipFont;  
        this.dicChartPanel[panelID].XTipFont_Pen.Color = fontColor;  
    }  
}
```

```
public void SetLeftYTipStyle(int panelID, Color backColor, Color fontColor, Font  
xTipFont)  
{  
    if (this.dicChartPanel.ContainsKey(panelID))  
    {  
        this.dicChartPanel[panelID].LeftyTip_Brush.Dispose();  
        this.dicChartPanel[panelID].LeftyTip_Brush = new SolidBrush(backColor);  
        this.dicChartPanel[panelID].LeftyTipFont_Brush.Dispose();  
        this.dicChartPanel[panelID].LeftyTipFont_Brush = new SolidBrush(fontColor);  
        this.dicChartPanel[panelID].LeftyTipFont = xTipFont;  
        this.dicChartPanel[panelID].LeftTipFont_Pen.Color = fontColor;  
    }  
}
```

```
public void SetRightYTipStyle(int panelID, Color backColor, Color fontColor, Font  
xTipFont)  
{  
    if (this.dicChartPanel.ContainsKey(panelID))  
    {  
        this.dicChartPanel[panelID].RightyTip_Brush.Dispose();  
        this.dicChartPanel[panelID].RightyTip_Brush = new SolidBrush(backColor);  
        this.dicChartPanel[panelID].RightyTipFont_Brush.Dispose();  
        this.dicChartPanel[panelID].RightyTipFont_Brush = new SolidBrush(fontColor);  
        this.dicChartPanel[panelID].RightyTipFont = xTipFont;  
        this.dicChartPanel[panelID].RightyTipFont_Pen.Color = fontColor;  
    }  
}
```

```
public void SetBackColor(int panelID, Color bgColor)  
{  
    if (this.dicChartPanel.ContainsKey(panelID))  
    {
```

```
this.dicChartPanel[panelID].BgBrush.Dispose();
this.dicChartPanel[panelID].BgBrush = new SolidBrush(bgColor);
}
}

public void SetBorderStyle(int panelID, Color borderColor, DashStyle dashStyle,
int width)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].BorderPen.Dispose();
        this.dicChartPanel[panelID].BorderPen = new Pen(borderColor, width);
        if (dashStyle != DashStyle.Custom)
        {
            this.dicChartPanel[panelID].BorderPen.DashStyle = dashStyle;
        }
    }
}

public void SetTitleStyle(int panelID, Color titleColor, Font titleFont)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].TitleFont_Brush.Dispose();
        this.dicChartPanel[panelID].TitleFont_Brush = new SolidBrush(titleColor);
        this.dicChartPanel[panelID].TitleFont = titleFont;
        this.dicChartPanel[panelID].TitleHeight =
this.CreateGraphics().MeasureString(" ", titleFont).Height + 5f;
    }
}

public void SetGridStyle(int panelID, Color gridColor, DashStyle dashStyle, bool
showGrid, int width)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].Grid_Pen.Color = gridColor;
        this.dicChartPanel[panelID].Grid_Pen.DashStyle = dashStyle;
        this.dicChartPanel[panelID].ShowGrid = showGrid;
    }
}

public void SetXScaleStyle(int panelID, Color scaleColor, Color fontColor, Font
scaleFont)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].XScalePen.Dispose();
        this.dicChartPanel[panelID].XScalePen = new Pen(scaleColor);
        this.dicChartPanel[panelID].CoordinateXFont_Brush.Dispose();
        this.dicChartPanel[panelID].CoordinateXFont_Brush = new SolidBrush(fontColor);
    }
}
```

```
this.dicChartPanel[panelID].CoordinateXFont = scaleFont;
this.dicChartPanel[panelID].ScaleX_Height =
(int)(this.CreateGraphics().MeasureString(" ", scaleFont).Height * 1.2) + 2;
}
}

public void SetLeftYScaleStyle(int panelID, Color scaleColor, Color fontColor,
Font scaleFont)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].LeftScalePen.Dispose();
        this.dicChartPanel[panelID].LeftScalePen = new Pen(scaleColor);
        this.dicChartPanel[panelID].LeftYFont_Brush.Dispose();
        this.dicChartPanel[panelID].LeftYFont_Brush = new SolidBrush(fontColor);
        this.dicChartPanel[panelID].LeftYFont = scaleFont;
    }
}

public void SetRightYScaleStyle(int panelID, Color scaleColor, Color fontColor,
Font scaleFont)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        this.dicChartPanel[panelID].RightScalePen.Dispose();
        this.dicChartPanel[panelID].RightScalePen = new Pen(scaleColor);
        this.dicChartPanel[panelID].RightYFont_Brush.Dispose();
        this.dicChartPanel[panelID].RightYFont_Brush = new SolidBrush(fontColor);
        this.dicChartPanel[panelID].RightYFont = scaleFont;
    }
}

public int AddChartPanel(int verticalPercent)
{
    int locationY = 0;
    foreach (ChartPanel cp in this.dicChartPanel.Values)
    {
        locationY += Convert.ToInt32((double)cp.VerticalPercent / 100 * this.Height);
    }
    int panelHeight = Convert.ToInt32((double)verticalPercent / 100 *
this.Height);
    ChartPanel chartPanel = new ChartPanel();
    chartPanel.VerticalPercent = verticalPercent;
    chartPanel.PanelID = CommonClass.GetPanelID();
    chartPanel.Bounds = new Rectangle(0, locationY, this.Width, panelHeight);
    this.dicChartPanel[chartPanel.PanelID] = chartPanel;
    return chartPanel.PanelID;
}

public void ResetNullGraph()
```

```

{
    ClearGraph();
    this.dicChartPanel.Clear();
    this.indExponentialMovingAverageList.Clear();
    this.indSimpleMovingAverageList.Clear();
    this.indMacdList.Clear();
    this.indBollList.Clear();
    this.indStochasticOscillatorList.Clear();
    this.dtAllMsg.Clear();
    this.dtAllMsg.Dispose();
    this.dtAllMsg = new DataTable();
    this.timekeyField = string.Empty;
    this.crossHair_y = -1;
    this.axisSpace = 8;
    this.leftPixSpace = 0;
    this.rightPixSpace = 0;
    this.showLeftScale = false;
    this.showRightScale = false;
    scrollLeftStep = 1;
    canDragSeries = false;
    vp_index = -1;
    drawValuePanelFlag = false;
    PaintGraph();
}

public void SetCandleTitleColor(string candleName, Color openColor, Color
highColor, Color lowColor, Color closeColor, int panelID)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        foreach (CandleSeries candleSeries in chartPanel.CandleSeriesList)
        {
            if (candleSeries.CandleName == candleName)
            {
                {
                    candleSeries.OpenTitleColor = openColor;
                    candleSeries.HighTitleColor = highColor;
                    candleSeries.LowTitleColor = lowColor;
                    candleSeries.CloseTitleColor = closeColor;
                }
            }
        }
    }
}

public void ShowValuePanel()
{
    if (selectedObject != null)
    {
        int curRecord = GetCrossOverIndex();
        if (JudgeSelectedSeries(curRecord, GetCrossHairPoint().Y, false) ==

```

```
selectedObject)
{
    vp_index = curRecord;
    PaintGraph();
}
}

public void checkMouseMoveLoop()
{
    int tick_dely = 5000000;
    while (true)
    {
        if (lastMouseMoveTime.AddTicks(tick_dely) <= DateTime.Now)
        {
            if (drawValuePanelFlag)
            {
                drawValuePanelFlag = !drawValuePanelFlag;
                if (selectedObject != null && this.IsHandleCreated)
                {
                    if (this.IsHandleCreated)
                    {
                        this.BeginInvoke(new ShowValuePanelDelegate(ShowValuePanel));
                    }
                }
            }
            Thread.Sleep(100);
        }
    }

    public CandleSeries AddCandle(string candleName, string openfield, string
highfield, string lowfield, string closefield, int panelID, bool displayTitleField)
    {
        if (openfield == null || highfield == null || lowfield == null || closefield
== null)
        {
            return null;
        }
        if (this.dtAllMsg.Columns.Contains(openfield) ||
this.dtAllMsg.Columns.Contains(highfield) ||
this.dtAllMsg.Columns.Contains(lowfield) ||
this.dtAllMsg.Columns.Contains(closefield))
        {
            return null;
        }
        if (this.dicChartPanel.ContainsKey(panelID))
        {
            ChartPanel chartPanel = this.dicChartPanel[panelID];
            CandleSeries candleSeries = new CandleSeries();
```



```
        candleSeries = new CandleSeries();
        candleSeries.CandleName = candleName;
        candleSeries.OpenField = openfield;
        candleSeries.HighField = highfield;
        candleSeries.LowField = lowfield;
        candleSeries.CloseField = closefield;
        chartPanel.YScaleField.AddRange(new string[] { openfield, highfield, lowfield,
closefield });
        candleSeries.Down_Color = Color.SkyBlue;
        candleSeries.Up_Color = Color.Red;
        candleSeries.DisplayTitleField = displayTitleField;
        DataColumn dcOpen = new DataColumn(openfield);
        DataColumn dcHigh = new DataColumn(highfield);
        DataColumn dcLow = new DataColumn(lowfield);
        DataColumn dcClose = new DataColumn(closefield);
        this.dtAllMsg.Columns.Add(dcOpen);
        this.dtAllMsg.Columns.Add(dcHigh);
        this.dtAllMsg.Columns.Add(dcLow);
        this.dtAllMsg.Columns.Add(dcClose);
        chartPanel.CandleSeriesList.Add(candleSeries);
        return candleSeries;
    }
    return null;
}

public void SetCandleStyle(string candleName, Color upColor, Color downColor,
Color middleColor)
{
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        foreach (CandleSeries cs in chartPanel.CandleSeriesList)
        {
            if (cs.CandleName == candleName)
            {
                cs.Up_Color = upColor;
                cs.Down_Color = downColor;
                cs.Middle_Color = middleColor;
            }
        }
    }
}

public void SetHistogramStyle(string field, Color upColor, Color downColor, int
width, bool lineStyle)
{
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        foreach (HistogramSeries hs in chartPanel.HistoramSeriesList)
        {
            if (hs.Field == field)
```

```
        {
            hs.LineStyle = lineStyle;
            hs.LineWidth = width;
            hs.UpLineColor = upColor;
            hs.DownLineColor = downColor;
            hs.LineStyle = lineStyle;
            return;
        }
    }
}

public void SetTrendLineStyle(string field, Color upLineColor, Color
downLineColor, int width, DashStyle dashStyle)
{
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        foreach (TrendLineSeries tls in chartPanel.TrendLineSeriesList)
        {
            if (tls.Field == field)
            {
                tls.UpLineColor = upLineColor;
                tls.UpLinePen.Width = width;
                tls.DownLineColor = downLineColor;
                tls.DownLinePen.Width = width;
                tls.UpLinePen.DashStyle = dashStyle;
                tls.DownLinePen.DashStyle = dashStyle;
                return;
            }
        }
    }
}

public HistogramSeries AddVOL(string field, string displayName, string
relateCandleName, int panelID)
{
    if (field == null || this.dtAllMsg.Columns.Contains(field))
    {
        return null;
    }
    else
    {
        if (this.dicChartPanel.ContainsKey(panelID))
        {
            ChartPanel chartPanel = this.dicChartPanel[panelID];
            HistogramSeries histogramSeries = new HistogramSeries();
            histogramSeries.UpLineColor = Color.Lime;
            histogramSeries.RelateCandleName = relateCandleName;
            histogramSeries.DownLineColor = Color.Lime;
            chartPanel.YScaleField.Add(field);
            histogramSeries.Field = field;
        }
    }
}
```

```
        histogramSeries.DisplayName = displayName;
        chartPanel.HistoramSeriesList.Add(histogramSeries);
        DataColumn dc = new DataColumn(field);
        this.dtAllMsg.Columns.Add(dc);
        return histogramSeries;
    }
    return null;
}

public TrendLineSeries AddTrendLine(string field, string displayName, int
panelID)
{
    if (field == null || this.dtAllMsg.Columns.Contains(field))
    {
        return null;
    }
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        TrendLineSeries lineSeries = new TrendLineSeries();
        lineSeries.UpLineColor = Color.Yellow;
        lineSeries.DownLineColor = Color.Yellow;
        lineSeries.Field = field;
        chartPanel.YScaleField.Add(field);
        if (displayName != null && displayName.Length > 0)
        {
            lineSeries.DisplayName = displayName;
        }
        chartPanel.TrendLineSeriesList.Add(lineSeries);
        DataColumn dc = new DataColumn(field);
        this.dtAllMsg.Columns.Add(dc);
        return lineSeries;
    }
    return null;
}

public IndicatorSimpleMovingAverage AddSMA(string field, string displayName,
string target, int cycle, int panelID)
{
    if (field == null || this.dtAllMsg.Columns.Contains(field))
    {
        return null;
    }
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        IndicatorSimpleMovingAverage indSMA = new IndicatorSimpleMovingAverage();
        indSMA.Target = target;
        indSMA.Cycle = cycle;
        indSMA.DataSource = this.dtAllMsg;
        indSMA.TrendLineSeries = AddTrendLine(field, displayName, panelID);
    }
}
```

```

        this.indSimpleMovingAverageList.Add(indSMA);
        return indSMA;
    }
    return null;
}

public IndicatorStochasticOscillator AddKDJ(string k, string d, string j, int
kPeriod,
    string close, string high, string low, int panelID)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        IndicatorStochasticOscillator indSO = new IndicatorStochasticOscillator();
        this.indStochasticOscillatorList.Add(indSO);
        indSO.DataSource = dtAllMsg;
        indSO.Close = close;
        indSO.High = high;
        indSO.Low = low;
        indSO.KPeriods = kPeriod;
        indSO.TlsK = AddTrendLine(k, k, panelID);
        SetTrendLineStyle(k, Color.White, Color.White, 1, DashStyle.Solid);
        indSO.TlsD = AddTrendLine(d, d, panelID);
        SetTrendLineStyle(d, Color.Yellow, Color.Yellow, 1, DashStyle.Solid);
        indSO.TlsJ = AddTrendLine(j, j, panelID);
        SetTrendLineStyle(j, Color.FromArgb(255, 0, 255), Color.FromArgb(255, 0, 255),
1, DashStyle.Solid);
        return indSO;
    }
    return null;
}

public IndicatorExponentialMovingAverage AddExponentialMovingAverage(string
field, string displayName, int cycle, string target, int panelID)
{
    if (field == null || this.dtAllMsg.Columns.Contains(field))
    {
        return null;
    }
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        IndicatorExponentialMovingAverage indEMA = new
IndicatorExponentialMovingAverage();
        indEMA.Cycle = cycle;
        indEMA.Target = target;
        indEMA.DataSource = this.dtAllMsg;
        indEMA.TrendLineSeries = AddTrendLine(field, displayName, panelID);
        this.indExponentialMovingAverageList.Add(indEMA);
        return indEMA;
    }
    return null;
}

```

```

    }
    public IndicatorMACD AddMACD(string macd, string diff, string dea, string close,
        int longCycle, int shortCycle, int signalPeriods, int panelID)
    {
        if (this.dicChartPanel.ContainsKey(panelID))
        {
            IndicatorMACD indicatorMACD = new IndicatorMACD();
            indicatorMACD.LongCycle = longCycle;
            indicatorMACD.ShortCycle = shortCycle;
            indicatorMACD.SignalPeriods = signalPeriods;
            string zeroLine = CommonClass.GetGuid();
            indicatorMACD.HsMACD = AddVOL(macd, macd, null, panelID);
            SetHistogramStyle(macd, Color.Red, Color.SkyBlue, 1, true);
            indicatorMACD.TlsDiff = AddTrendLine(diff, diff, panelID);
            SetTrendLineStyle(diff, Color.White, Color.White, 1, DashStyle.Solid);
            indicatorMACD.TlsDea = AddTrendLine(dea, dea, panelID);
            SetTrendLineStyle(dea, Color.Yellow, Color.Yellow, 1, DashStyle.Solid);
            indicatorMACD.Close = close;
            this.dtAllMsg.Columns.Add(indicatorMACD.LongCycleEMA);
            this.dtAllMsg.Columns.Add(indicatorMACD.ShortCycleEMA);
            indicatorMACD.DataSource = this.dtAllMsg;
            this.indMacdList.Add(indicatorMACD);
            return indicatorMACD;
        }
        return null;
    }

    public IndicatorBollingerBands AddBOLL(string mid, string up, string down, string
close,
        int periods, int standardDeviations, int panelID)
    {
        if (this.dicChartPanel.ContainsKey(panelID))
        {
            IndicatorBollingerBands indBoll = new IndicatorBollingerBands();
            indBoll.TlsM = AddTrendLine(mid, mid, panelID);
            SetTrendLineStyle(mid, Color.White, Color.White, 1, DashStyle.Solid);
            indBoll.TlsU = AddTrendLine(up, up, panelID);
            SetTrendLineStyle(up, Color.Yellow, Color.Yellow, 1, DashStyle.Solid);
            indBoll.TlsD = AddTrendLine(down, down, panelID);
            SetTrendLineStyle(down, Color.FromArgb(255, 0, 255), Color.FromArgb(255, 0, 255),
1, DashStyle.Solid);
            indBoll.Close = close;
            indBoll.DataSource = this.dtAllMsg;
            indBoll.Periods = periods;
            indBoll.StandardDeviations = standardDeviations;
            this.indBollList.Add(indBoll);
            return indBoll;
        }
        return null;
    }

    public void ResetCrossOverRecord()

```

```
{
    if (this.dtAllMsg.Rows.Count >= this.GetMaxVisibleRecord())
    {
        if (this.dtAllMsg.Rows.Count > 0 && showCrossLine)
        {
            if (this.crossOverIndex < firstVisibleRecord - 1)
            {
                this.crossOverIndex = firstVisibleRecord - 1;
            }
            if (this.crossOverIndex > LastVisibleRecord - 1)
            {
                this.crossOverIndex = LastVisibleRecord - 1;
            }
        }
    }
}

private void ScrollLeft(int step)
{
    if (this.dtAllMsg.Rows.Count > 1 && firstVisibleRecord > 1)
    {
        if (this.dtAllMsg.Rows.Count > GetMaxVisibleRecord())
        {
            if (firstVisibleRecord - step >= 1)
            {
                firstVisibleRecord = firstVisibleRecord - step;
                LastVisibleRecord = LastVisibleRecord - step;
            }
            else
            {
                LastVisibleRecord = LastVisibleRecord - firstVisibleRecord;
                firstVisibleRecord = 1;
            }
        }
    }
}

private void ScrollRight(int step)
{
    if (this.dtAllMsg.Rows.Count > 1 && LastVisibleRecord <
this.dtAllMsg.Rows.Count)
    {
        if (this.dtAllMsg.Rows.Count > GetMaxVisibleRecord())
        {
            if (LastVisibleRecord + step > this.dtAllMsg.Rows.Count)
            {
                firstVisibleRecord = firstVisibleRecord + (this.dtAllMsg.Rows.Count -
LastVisibleRecord);
                LastVisibleRecord = this.dtAllMsg.Rows.Count;
            }
            else
            {

```

```
        firstVisibleRecord = firstVisibleRecord + step;
        LastVisibleRecord = LastVisibleRecord + step;
    }
}
}
}
private void ZoomIn(int step)
{
    if (this.axisSpace < 50)
    {
        int oriMax = GetMaxVisibleRecord();
        bool dealWith = false;
        if (this.dtAllMsg.Rows.Count < oriMax)
        {
            dealWith = true;
        }
        this.axisSpace = this.axisSpace + 1;
        int nowMax = GetMaxVisibleRecord();
        int subRecord = oriMax - nowMax;
        if (this.dtAllMsg.Rows.Count >= nowMax)
        {
            if (dealWith)
            {
                firstVisibleRecord = 1;
                LastVisibleRecord = nowMax;
            }
            else
            {
                this.firstVisibleRecord = this.firstVisibleRecord + subRecord / 2;
                this.LastVisibleRecord = this.firstVisibleRecord + nowMax - 1;
            }
        }
    }
}
private void ZoomOut(int step)
{
    if (this.axisSpace > 1)
    {
        int oriMax = GetMaxVisibleRecord();
        this.axisSpace = this.axisSpace - 1;
        int nowMax = GetMaxVisibleRecord();
        int subRecord = nowMax - oriMax;
        int f = this.firstVisibleRecord - subRecord / 2;
        int l = this.LastVisibleRecord + (subRecord - subRecord / 2);
        if (f < 1 && l > this.dtAllMsg.Rows.Count)
        {
            firstVisibleRecord = 1;
            LastVisibleRecord = this.dtAllMsg.Rows.Count;
        }
        else if (f < 1)
```

```

    {
        LastVisibleRecord = LastVisibleRecord + subRecord - (firstVisibleRecord - 1);
        firstVisibleRecord = 1;
        if (LastVisibleRecord > this.dtAllMsg.Rows.Count)
        {
            LastVisibleRecord = this.dtAllMsg.Rows.Count;
        }
    }
    else if (1 > this.dtAllMsg.Rows.Count)
    {
        firstVisibleRecord = firstVisibleRecord - (subRecord -
(this.dtAllMsg.Rows.Count - this.LastVisibleRecord));
        LastVisibleRecord = this.dtAllMsg.Rows.Count;
        if (firstVisibleRecord < 1)
        {
            firstVisibleRecord = 1;
        }
    }
    else
    {
        this.firstVisibleRecord = f;
        this.LastVisibleRecord = 1;
    }
}

public void SetXScaleField(string field)
{
    if (this.timekeyField == null || this.timekeyField.Length == 0)
    {
        this.timekeyField = field;
        DataColumn dcTimekey = new DataColumn(timekeyField);
        this.dtAllMsg.Columns.Add(dcTimekey);
        this.dtAllMsg.PrimaryKey = new DataColumn[] { dcTimekey };
    }
}

public void SetYScaleField(int panelID, string[] field)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        ChartPanel chartPanel = this.dicChartPanel[panelID];
        chartPanel.YScaleField.Clear();
        foreach (string ch in field)
        {
            chartPanel.YScaleField.Add(ch);
        }
    }
}

public void RefreshGraph()
{
    InitFirstAndLastVisibleRecord();
}

```



```
        setVisibleExtremeValue();
        ResetCrossOverRecord();
        PaintGraph();
    }

    public void ClearGraph()
    {
        this.dtAllMsg.Clear();
        this.firstVisibleRecord = 0;
        this.LastVisibleRecord = 0;
        showCrossLine = false;
        RefreshGraph();
    }

    public void InitFirstAndLastVisibleRecord()
    {
        if (this.dtAllMsg.Rows.Count == 0)
        {
            this.firstVisibleRecord = 0;
            this.LastVisibleRecord = 0;
        }
        else
        {
            int maxVisibleRecord = GetMaxVisibleRecord();
            if (this.dtAllMsg.Rows.Count < maxVisibleRecord)
            {
                firstVisibleRecord = 1;
                this.LastVisibleRecord = this.dtAllMsg.Rows.Count;
            }
            else
            {
                if (firstVisibleRecord != 0 && LastVisibleRecord != 0
                    && !lastRecordIsVisible)
                {
                    DataRow dr = this.dtAllMsg.Rows.Find(lastVisibleTimeKey);
                    if (dr != null)
                    {
                        int index = this.dtAllMsg.Rows.IndexOf(dr);
                        LastVisibleRecord = index + 1;
                        firstVisibleRecord = LastVisibleRecord - maxVisibleRecord + 1;
                        if (firstVisibleRecord < 1)
                        {
                            firstVisibleRecord = 1;
                        }
                    }
                }
                else
                {
                    LastVisibleRecord = this.dtAllMsg.Rows.Count;
                    firstVisibleRecord = LastVisibleRecord - maxVisibleRecord + 1;
                    if (firstVisibleRecord < 1)
```

```

    {
firstVisibleRecord = 1;
    }
    }
}
}
}
private void setVisibleExtremeValue()
{
    if (GetWorkSpaceX() > 0)
    {
        int firstR = firstVisibleRecord - 1;
        int lastR = LastVisibleRecord;
        foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
        {
            Dictionary<CandleSeries, List<object[]>> kValueList = new
Dictionary<CandleSeries, List<object[]>>();
            List<double> valueList = new List<double>();
            if (this.dtAllMsg.Rows.Count > 0)
            {
                for (int i = firstR; i < lastR; i++)
                {
                    string timeKey = this.dtAllMsg.Rows[i][timekeyField].ToString();
                    DataRow dr = dtAllMsg.Rows[i];
                    foreach (string field in chartPanel.YScaleField)
                    {
                        double fieldValue = 0;
                        Double.TryParse(dr[field].ToString(), out fieldValue);
                        valueList.Add(fieldValue);
                    }
                }
            }
            foreach (CandleSeries cs in chartPanel.CandleSeriesList)
            {
                if (!kValueList.ContainsKey(cs))
                {
                    kValueList.Add(cs, new List<object[]>());
                }
                double open = 0;
                Double.TryParse(dr[cs.OpenField].ToString(), out open);
                double high = 0;
                Double.TryParse(dr[cs.HighField].ToString(), out high);
                double low = 0;
                Double.TryParse(dr[cs.LowField].ToString(), out low);
                double close = 0;
                Double.TryParse(dr[cs.CloseField].ToString(), out close);
                kValueList[cs].Add(new object[] { i, open });
                kValueList[cs].Add(new object[] { i, high });
                kValueList[cs].Add(new object[] { i, low });
                kValueList[cs].Add(new object[] { i, close });
                if (chartPanel.YScaleField.Count == 0)
                {

```

```
valueList.Add(open);
valueList.Add(high);
valueList.Add(low);
valueList.Add(close);
    }
    else
    {
if (chartPanel.YScaleField.Contains(cs.OpenField))
{
    valueList.Add(open);
}
if (chartPanel.YScaleField.Contains(cs.HighField))
{
    valueList.Add(high);
}
if (chartPanel.YScaleField.Contains(cs.LowField))
{
    valueList.Add(low);
}
if (chartPanel.YScaleField.Contains(cs.CloseField))
{
    valueList.Add(close);
}
    }
}
foreach (HistogramSeries hs in chartPanel.HistoramSeriesList)
{
    double volume = 0;
    Double.TryParse(dr[hs.Field].ToString(), out volume);
    if (chartPanel.YScaleField.Count == 0)
    {
valueList.Add(0);
valueList.Add(volume);
    }
    else
    {
if (chartPanel.YScaleField.Contains(hs.Field))
{
    valueList.Add(0);
    valueList.Add(volume);
}
    }
}
foreach (TrendLineSeries tls in chartPanel.TrendLineSeriesList)
{
    double lineValue = 0;
    Double.TryParse(dr[tls.Field].ToString(), out lineValue);
    if (chartPanel.YScaleField.Count == 0)
    {
valueList.Add(lineValue);
```

```

    }
    else
    {
if (chartPanel.YScaleField.Contains(tls.Field))
{
    valueList.Add(lineValue);
}
    }
}
    }

    chartPanel.MaxValue = CommonClass.GetHighValue(valueList);
    chartPanel.MinValue = CommonClass.GetLowValue(valueList);
    foreach (CandleSeries cs in kValueList.Keys)
    {
        cs.MaxRecord = CommonClass.GetHighRecord(kValueList[cs]);
        cs.MinRecord = CommonClass.GetLowRecord(kValueList[cs]);
    }
    }
}

public void CalcutaIndicator(object indicator,int startIndex,int endIndex)
{
    for (int i = startIndex; i <= endIndex; i++)
    {
        DataRow dr = this.dtAllMsg.Rows[i];
        DateTime                dateTime                =
CommonClass.GetDateTimeByTimeKey(dr[timekeyField].ToString());
        if (indicator is IndicatorSimpleMovingAverage)
        {
            IndicatorSimpleMovingAverage    indSMA        =        indicator        as
IndicatorSimpleMovingAverage;
            double smaValue = indSMA.Calculate(i);
            if (smaValue != NULL)
            {
                SetValue(indSMA.TrendLineSeries.Field, smaValue, dateTime);
            }
        }
        else if (indicator is IndicatorExponentialMovingAverage)
        {
            IndicatorExponentialMovingAverage    indEMA        =        indicator        as
IndicatorExponentialMovingAverage;
            double emaValue = indEMA.Calculate(i);
            if (emaValue != NULL)
            {
                SetValue(indEMA.TrendLineSeries.Field, emaValue, dateTime);
            }
        }
        else if (indicator is IndicatorStochasticOscillator)
        {

```

```

        IndicatorStochasticOscillator indSO = indicator as
IndicatorStochasticOscillator;
        if (!(dr[indSO.High] is DBNull) && !(dr[indSO.Low] is DBNull)
&& !(dr[indSO.Close] is DBNull))
        {
            double[] kdj = indSO.Calculate(i);
            SetValue(indSO.TlsK.Field, kdj[0], dateTime);
            SetValue(indSO.TlsD.Field, kdj[1], dateTime);
            SetValue(indSO.TlsJ.Field, kdj[2], dateTime);
        }
    }
    else if (indicator is IndicatorMACD)
    {
        IndicatorMACD indMacd = indicator as IndicatorMACD;
        if (!(dr[indMacd.Close] is DBNull))
        {
            double shortEMA =
CommonClass.CalculateExponentialMovingAvg(indMacd.ShortCycleEMA, indMacd.Close,
indMacd.ShortCycle, indMacd.DataSource, i);
            double longEMA =
CommonClass.CalculateExponentialMovingAvg(indMacd.LongCycleEMA, indMacd.Close,
indMacd.LongCycle, indMacd.DataSource, i);
            SetValue(indMacd.ShortCycleEMA, shortEMA, dateTime);
            SetValue(indMacd.LongCycleEMA, longEMA, dateTime);
            double[] macdValue = indMacd.Calculate(i);
            SetValue(indMacd.HsMACD.Field, macdValue[0], dateTime);
            SetValue(indMacd.TlsDiff.Field, macdValue[1], dateTime);
            SetValue(indMacd.TlsDea.Field, macdValue[2], dateTime);
        }
    }
    else if (indicator is IndicatorBollingerBands)
    {
        IndicatorBollingerBands indBoll = indicator as IndicatorBollingerBands;
        double[] bollValue = indBoll.Calculate(i);
        SetValue(indBoll.TlsM.Field, bollValue[0], dateTime);
        SetValue(indBoll.TlsU.Field, bollValue[1], dateTime);
        SetValue(indBoll.TlsD.Field, bollValue[2], dateTime);
    }
}

public void SetValue(string fieldName, object value, DateTime dateTime)
{
    if (!dtAllMsg.Columns.Contains(fieldName))
    {
        dtAllMsg.Columns.Add(fieldName);
    }
    string timeKey = CommonClass.GetTimeKey(dateTime);
    DataRow dr = dtAllMsg.Rows.Find(timeKey);
    if (dr != null)
    {

```

```

        dr[fieldName] = value;
    }
    else
    {
        DateTime dtEnd = DateTime.Now;
        bool flag = false;
        if (this.dtAllMsg.Rows.Count > 0)
        {
            flag = true;
            dtEnd =
CommonClass.GetDateTimeByTimeKey(this.dtAllMsg.Rows[this.dtAllMsg.Rows.Count -
1][timekeyField].ToString());
        }
        dr = dtAllMsg.NewRow();
        dr[timekeyField] = timeKey;
        dr[fieldName] = value;
        this.dtAllMsg.Rows.Add(dr);
        if (flag && dateTime < dtEnd)
        {
            DataView dv = dtAllMsg.DefaultView;
            dv.Sort = timekeyField + " ASC";
            this.dtAllMsg = dv.ToTable();
            this.dtAllMsg.PrimaryKey = new DataColumn[]
{ this.dtAllMsg.Columns[timekeyField] };
        }
    }
    int r = this.dtAllMsg.Rows.IndexOf(dr);
    bool ind_flag = false;
    for (int m = r; m < this.dtAllMsg.Rows.Count; m++)
    {
        if (this.indSimpleMovingAverageList.Count > 0)
        {
            foreach (IndicatorSimpleMovingAverage indSMA in
this.indSimpleMovingAverageList)
            {
                if (indSMA.Target == fieldName)
                {
                    ind_flag = true;
                    CalcutaIndicator(indSMA, m, m);
                }
            }
        }
        if (this.indExponentialMovingAverageList.Count > 0)
        {
            foreach (IndicatorExponentialMovingAverage indEMA in
this.indExponentialMovingAverageList)
            {
                if (indEMA.Target == fieldName)
                {
                    ind_flag = true;

```

```

    CalcutaIndicator(indEMA, m, m);
    }
    }
    }
    if (this.indStochasticOscillatorList.Count > 0)
    {
        foreach (IndicatorStochasticOscillator indSO in
this.indStochasticOscillatorList)
        {
            if (fieldName == indSO.High || fieldName == indSO.Low || fieldName ==
indSO.Close)
            {
ind_flag = true;
CalcutaIndicator(indSO, m, m);
            }
        }
    }
    if (this.indMacdList.Count > 0)
    {
        foreach (IndicatorMACD indMacd in this.indMacdList)
        {
            if (fieldName == indMacd.Close)
            {
ind_flag = true;
CalcutaIndicator(indMacd, m, m);
            }
        }
    }
    if (this.indBollList.Count > 0)
    {
        foreach (IndicatorBollingerBands indBOLL in this.indBollList)
        {
            if (fieldName == indBOLL.Close)
            {
ind_flag = true;
CalcutaIndicator(indBOLL, m, m);
            }
        }
    }
    if (!ind_flag)
    {
        break;
    }
}

public int GetCurrentPanel()
{
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        if (crossHair_y >= chartPanel.Bounds.Y && crossHair_y <= chartPanel.Bounds.Y +

```

```
chartPanel.Bounds.Height)
{
    return chartPanel.PanelID;
}
return -1;
}

public float GetValueYPixel(CharPanel chartPanel, double chartValue)
{
    return Convert.ToSingle((chartPanel.MaxValue - chartValue) /
(chartPanel.MaxValue - chartPanel.MinValue) * GetWorkSpaceY(chartPanel.PanelID)
+ chartPanel.TitleHeight + chartPanel.Bounds.Y);
}

public int GetMaxVisibleRecord()
{
    if (this.axisSpace == 0)
    {
        return this.GetWorkSpaceX();
    }
    else
    {
        return this.GetWorkSpaceX() / this.AxisSpace;
    }
}

public Point GetCrossHairPoint()
{
    int x = this.PointToClient(MousePosition).X;
    int y = this.PointToClient(MousePosition).Y;
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        if (y >= chartPanel.Bounds.Y && y <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height)
        {
            if (y > this.Height - chartPanel.ScaleX_Height)
            {
                y = this.Height - chartPanel.ScaleX_Height;
            }
            if (showLeftScale)
            {
                if (x < leftPixSpace)
                {
x = leftPixSpace;
                }
            }
            if (showRightScale)
            {
                if (x > this.Width - rightPixSpace)
                {
x = this.Width - rightPixSpace;
                }
            }
        }
    }
}
```



```

        }
        break;
    }
}
return new Point(x, y);
}
public int GetCrossOverIndex()
{
    Point mousePoint = GetCrossHairPoint();
    return (mousePoint.X - this.LeftPixSpace) / this.AxisSpace +
this.firstVisibleRecord - 1;
}
public double GetCurrentValue()
{
    Point mouseP = GetCrossHairPoint();
    foreach (ChartPanel chartPanel in this.dicChartPanel.Values)
    {
        if (crossHair_y >= chartPanel.Bounds.Y && crossHair_y <= chartPanel.Bounds.Y +
chartPanel.Bounds.Height)
        {
            if (GetWorkSpaceY(chartPanel.PanelID) > 0)
            {
                double everyPointValue = (chartPanel.MaxValue - chartPanel.MinValue) /
GetWorkSpaceY(chartPanel.PanelID);
                return chartPanel.MaxValue - (crossHair_y - chartPanel.TitleHeight -
chartPanel.Bounds.Y) * everyPointValue;
            }
        }
    }
    return 0;
}
public int GetWorkSpaceX()
{
    return this.Width - this.LeftPixSpace - this.RightPixSpace;
}
public int GetWorkSpaceY(int panelID)
{
    if (this.dicChartPanel.ContainsKey(panelID))
    {
        return dicChartPanel[panelID].Bounds.Height -
dicChartPanel[panelID].ScaleX_Height - (int)dicChartPanel[panelID].TitleHeight;
    }
    else
    {
        return 0;
    }
}
public void AddMA(int panelID, string candleName, string buyEMA, string buyTarget,
string selleMA, string sellTarget)
{

```

```

        this.AddExponentialMovingAverage(buyEMA, buyEMA, 6, buyTarget, panelID);
        this.SetTrendLineStyle(buyEMA, Color.SkyBlue, Color.Red, 1,
DashStyle.Solid);
        this.AddExponentialMovingAverage(sellEMA, sellEMA, 5, sellTarget, panelID);
        this.SetTrendLineStyle(sellEMA, Color.Yellow, Color.Yellow, 1,
DashStyle.Solid);
        this.SetCandleBuySellField(candleName, buyEMA, sellEMA);
    }
    public void ChoseIndicator(IndicatorType indicatorType)
    {
    }
    #endregion
}

public class CandleSeries
{
    public CandleSeries()
    {
    }

    ~CandleSeries()
    {
        if (this.upLine_Brush != null) upLine_Brush.Dispose();
        if (this.downLine_Brush != null) downLine_Brush.Dispose();
        if (this.middleLine_Pen != null) middleLine_Pen.Dispose();
        if (this.upLine_Pen != null) upLine_Pen.Dispose();
        if (this.upLine_TransparentBrush != null)
upLine_TransparentBrush.Dispose();
        if (this.downLine_Pen != null) downLine_Pen.Dispose();
    }
    private Font bsFont = new Font("", 12, FontStyle.Bold);
    public Font BsFont
    {
        get { return bsFont; }
        set { bsFont = value; }
    }
    private Color buyColor = Color.Red;
    public Color BuyColor
    {
        get { return buyColor; }
        set { buyColor = value; }
    }
    private Color sellColor = Color.SkyBlue;
    public Color SellColor
    {
        get { return sellColor; }
        set { sellColor = value; }
    }
    private string buyText = "B";
    public string BuyText
    {

```

```
        get { return buyText; }
        set { buyText = value; }
    }
    private string sellText = "S";
    public string SellText
    {
        get { return sellText; }
        set { sellText = value; }
    }
    private string candleName;
    public string CandleName
    {
        get { return candleName; }
        set { candleName = value; }
    }
    private string[] indBuySellField = new string[2];
    public string[] IndBuySellField
    {
        get { return indBuySellField; }
        set { indBuySellField = value; }
    }
    private string openField;
    public string OpenField
    {
        get { return openField; }
        set { openField = value; }
    }
    private string closeField;
    public string CloseField
    {
        get { return closeField; }
        set { closeField = value; }
    }
    private string lowField;
    public string LowField
    {
        get { return lowField; }
        set { lowField = value; }
    }
    private bool hasSelect = false;
    public bool HasSelect
    {
        get { return hasSelect; }
        set { hasSelect = value; }
    }
    private int maxRecord = 0;
    public int MaxRecord
    {
        get { return maxRecord; }
        set { maxRecord = value; }
```

```
    }
    private int minRecord = 0;
    public int MinRecord
    {
        get { return minRecord; }
        set { minRecord = value; }
    }
    private Brush upLine_Brush;
    public Brush UpLine_Brush
    {
        get { return upLine_Brush; }
        set { upLine_Brush = value; }
    }
    private Brush downLine_Brush;
    public Brush DownLine_Brush
    {
        get { return downLine_Brush; }
        set { downLine_Brush = value; }
    }
    private Pen middleLine_Pen;
    public Pen MiddleLine_Pen
    {
        get { return middleLine_Pen; }
        set { middleLine_Pen = value; }
    }
    private Pen upLine_Pen;
    public Pen UpLinePen
    {
        get { return upLine_Pen; }
        set { upLine_Pen = value; }
    }
    private Pen downLine_Pen;
    public Pen DownLine_Pen
    {
        get { return downLine_Pen; }
        set { downLine_Pen = value; }
    }
    private Brush upLine_TransparentBrush;
    public Brush UpLineTransparentBrush
    {
        get { return upLine_TransparentBrush; }
        set { upLine_TransparentBrush = value; }
    }
}
```